

C++面向对象课程设计报告

校园RPG冒险游戏

专 业：计算机科学与技术

课程名称：C++面向对象程序设计

项目名称：校园RPG冒险游戏

开发环境：C++17 + Win32 API + MinGW

开发工具：VS Code / CMake / GCC 6.3.0

2026年7月

目 录

- 一、项目简介
- 二、项目特色与创新
- 三、项目管理
- 四、需求分析
- 五、系统设计
- 六、类设计与UML图
- 七、关键功能实现
- 八、测试与结果分析
- 九、总结与展望
- 十、参考文献

一、项目简介

项目名称：校园RPG冒险游戏（Campus Adventure RPG）

本项目是一个基于C++面向对象编程的Windows GUI校园冒险RPG游戏。游戏以校园生活为冒险背景，构建轻松休闲的角色扮演冒险世界。玩家扮演一名校园冒险者，在校园场景中探索、遭遇校园怪物、完成师生发布的各类任务、收集道具、购买物资、提升自身等级与属性，最终完成校园冒险成长。

项目采用C++17标准进行开发，综合运用了类继承、多态、虚函数、STL容器、文件持久化、多线程自动存档、Win32 GUI编程等核心技术。系统由六大核心模块组成：角色管理模块、背包管理模块、商店系统模块、任务系统模块、战斗系统模块和等级成长模块，完整实现了从一个RPG游戏从角色创建到最终成长的全流程功能。

GitHub仓库地址：

```
https://github.com/yourusername/CampusAdventureRPG ████████████████████
```

技术架构：

层次	技术	说明
语言标准	C++17	使用现代C++特性（auto、range-for等）
GUI框架	Win32 API	原生Windows窗口、对话框、资源文件
构建系统	CMake + MinGW	跨平台构建配置，支持UTF-8编码
数据持久化	文件I/O	玩家数据、任务状态存档与读取
多线程	Win32 CreateThread	后台自动定时存档（每60秒）
设计模式	工厂/策略/模板方法	SkillFactory、Item多态体系

二、项目特色与创新

本项目在满足课程基本要求的基础上，完成了多项挑战任务，充分展示了面向对象编程的深度应用。以下表格列出了完成的所有挑战任务及其实现情况：

序号	挑战任务	状态	技术要点
1	多态继承体系：Item基类 -> Food/Medicine/Equipment三个派生类	完成	虚函数use()/clone()/getTypeName()实现运行时多态
2	技能系统：Skill基类 -> Melee/Magic/Heal/Debuff四类派生	完成	工厂模式、技能升级、MP消耗、伤害计算策略
3	Win32 GUI图形界面：自定义对话框和窗口	完成	9个按钮、8个对话框、微软雅黑字体渲染
4	文件持久化存档：角色/背包/任务全部支持存读档	完成	自定义序列化格式、管道符分隔、状态恢复
5	多线程自动存档：后台线程定时保存	完成	Win32 CreateThread，每60秒自动保存
6	回合制战斗系统：玩家与怪物轮流出招	完成	随机伤害浮动、攻防计算、3种梯度怪物
7	等级自动成长：EXP达标自动升级	完成	HP+20/级、ATK+5/级、溢出经验保留
8	任务进度追踪：击杀计数联动任务	完成	4个校园主题任务、状态机流转
9	装备穿戴/卸下：永久属性加成	完成	武器/防具/饰品三槽位、属性加减
10	UTF-8编码方案解决中文乱码	完成	CP_UTF8编码页 + -fexec-charset=UTF-8

三、项目管理

3.1 团队分工

角色	职责	负责模块
成员A	项目架构 + 核心逻辑	Item多态体系、背包系统、商店系统、CMake构建
成员B	GUI开发 + 战斗系统	Win32窗口框架、对话框设计、回合制战斗、怪物设计
成员C	角色系统 + 数据持久化	角色创建、等级成长、存档系统、多线程自动存档
成员D	任务系统 + 技能系统	任务模板、进度追踪、技能树、文档撰写

3.2 项目进度安排

阶段	时间	任务内容	交付物
第一阶段	第1-2周	需求分析、系统设计、UML建模	需求文档、UML类图
第二阶段	第3-4周	核心类实现：Item多态体系、背包、角色	可编译运行的框架代码
第三阶段	第5-6周	进阶功能：商店、战斗、任务、技能	完整控制台版本
第四阶段	第7-8周	GUI开发：Win32窗口、对话框、资源文件	GUI可执行程序
第五阶段	第9-10周	存档系统、多线程、调试编码问题	含存档功能的完整版本
第六阶段	第11-12周	测试、文档、报告撰写	测试报告、课程设计报告PDF

四、需求分析

4.1 功能需求

系统要实现六大核心功能模块，每个模块包含完整的CRUD操作：

模块	功能点	关键需求
角色管理	创建角色、查看信息、属性管理、存读档	名称、等级Lv1-50、HP/MP、EXP、金币、STR/VIT/AGI/INT
背包管理	获得物品、查看背包、使用物品、删除物品	3类12种道具、无限容量、堆叠管理
商店系统	查看商品、购买、出售、金币结算	12种商品固定售卖、半价回收
任务系统	查看、接受、完成、领奖	4个梯度任务、击杀计数、状态机流转
战斗系统	选敌、攻击、血量变化、结果判定	3种校园怪物、回合制、EXP+金币
等级成长	经验累计、自动升级、属性增长	EXP阈值公式、HP+20/ATK+5每级

4.2 非功能需求

- 性能：GUI界面响应时间小于500ms，战斗结算小于200ms
- 可靠性：多线程自动存档防止数据丢失，存档文件校验
- 可用性：微软雅黑字体渲染，中文界面，矩形按钮交互
- 可维护性：头文件与实现分离，清晰的命名空间与注释规范
- 可扩展性：Item/Skill基类支持派生扩展新类型
- 兼容性：Windows 7/10/11，MinGW GCC 6.3.0+编译

五、系统设计

5.1 系统架构

系统采用分层架构设计：表示层（Win32 GUI）-> 业务逻辑层（核心类）-> 数据持久层（文件I/O）。三层之间通过清晰的接口解耦，表示层通过全局变量访问业务逻辑对象，数据层通过序列化接口与业务层交互。

层次	组件	职责
表示层	main.cpp, resources.rc	Win32窗口/对话框、按钮事件分发、信息显示
业务层	Player, Inventory, Shop, TaskSystem, Skill	游戏核心逻辑、数据计算、状态管理
数据层	SaveManager, 文件I/O	序列化/反序列化、文件读写、自动存档线程

5.2 模块划分

角色管理模块（BaseCharacter + Player）

负责角色创建、属性管理、等级成长。BaseCharacter提供基础属性与战斗接口；Player继承并扩展了经验值、金币、技能、背包引用等玩家专属属性。

道具系统模块（Item -> Food/Medicine/Equipment）

采用多态继承体系。Item为抽象基类，定义use()/clone()/getTypeName()纯虚函数。Food恢复HP，Medicine高额回血，Equipment永久加成属性。

背包系统模块（Inventory）

使用vector管理物品指针，支持增删查改、堆叠管理、序列化存档。

商店系统模块（Shop）

管理12种商品模板，提供购买（clone商品）和出售（半价回收）功能。

任务系统模块（Task + TaskSystem）

Task封装单个任务状态机，TaskSystem管理任务列表、文件加载、状态流转。

战斗系统模块（main.cpp + EnemyInfo结构体）

三种梯度怪物，回合制自动战斗，击杀计数联动任务系统。

5.3 数据流设计

玩家操作 -> GUI按钮事件 -> 弹出对话框 -> 修改业务对象 ->
Refresh()更新主窗口显示。战斗/任务完成后自动触发经验累积 ->
checkLevelUp()检测升级 ->
属性刷新。存档线程每60秒自动调用SaveManager序列化当前状态 ->
写入player_save.txt文件。

六、类设计与UML图

6.1 核心类层次结构

项目共设计了约15个核心类，形成了清晰的继承与组合关系。以下为UML类图说明：

6.1.1 角色类继承体系

BaseCharacter（抽象基类）：定义角色通用属性（name/level/str/vit/agi/intl/hp/mp/def/mdef）和虚函数接口。Player继承BaseCharacter，扩展了exp/gold/skillPoint等玩家独有属性，并添加了技能管理、背包引用、任务列表等容器成员。

6.1.2 物品类多态体系

Item（抽象基类）-> Food（食物，恢复HP）-> Medicine（药品，高额回血）-> Equipment（装备，永久属性加成）。通过虚函数实现运行时多态：背包统一存储Item*指针，调用use()时自动分派到正确的派生类实现。

6.1.3 技能类多态体系

Skill（抽象基类）-> MeleeSkill（物理伤害）-> MagicSkill（魔法伤害）-> HealSkill（治疗）-> DebuffSkill（减益）。SkillFactory工厂类管理所有技能模板，使用原型模式clone()创建技能实例。

6.1.4 管理类组合关系

Inventory聚合Item*（vector容器）；Shop聚合Item*（商品模板）；TaskSystem聚合Task（vector容器）；Player关联Skill*（已学技能列表）。这些管理类通过组合关系持有被管理对象的指针或实例。

6.2 UML类图（Mermaid格式）

```
classDiagram
    class BaseCharacter { <> ... }
    class Player { +exp, gold, skillPoint +learnSkill() }
    class Item { <> +use()* +clone()* }
    class Food { +hpRestore }
    class Medicine { +hpRestore }
    class Equipment { +attackBonus +defenseBonus }
    class Inventory { +items: vector~Item~ }
    class Shop { +shopItems: vector~Item~ }
    class Task { +status +accept() }
    class TaskSystem { +tasks: vector~Task~ }
    class Skill { <> +calcDamage()* }
    class MeleeSkill / MagicSkill / HealSkill / DebuffSkill
    class SaveManager { +savePlayer() +loadPlayer() }
    BaseCharacter <|-- Player
    Item <|-- Food
    Item <|-- Medicine
    Item <|-- Equipment
    Skill <|-- MeleeSkill
    Skill <|-- MagicSkill
    Skill <|-- HealSkill
    Skill <|-- DebuffSkill
    Inventory o-- Item
    Shop o-- Item
    TaskSystem o-- Task
    Player o-- Skill
    SaveManager ..> Player
```

图1：核心类UML类图

七、关键功能实现

7.1 物品多态体系实现

物品系统采用经典的工厂+策略模式。Item基类定义纯虚函数接口，三个派生类各自实现不同的use()策略。商店购买时通过clone()创建物品副本存入背包，背包使用基类指针统一管理，调用use()时自动多态分派。

核心代码——Item基类定义：

```
class Item { protected: int m_id; std::string m_name; ItemType m_type; int m_price; public: virtual void use(RPG::BaseCharacter& target) = 0; virtual Item* clone() const = 0; virtual std::string getTypeName() const = 0; }
```

7.2 回合制战斗系统

战斗系统在main.cpp中实现，使用EnemyInfo结构体存储怪物属性。战斗采用while循环模拟回合制，双方轮流攻击，实时计算伤害（含随机浮动和防御减免），结算后根据胜负发放奖励并更新击杀计数。战斗核心逻辑约50行代码，支持最多50回合上限防止死循环。

7.3 自动存档多线程

由于MinGW g++ 6.3.0不完全支持std::thread，使用Win32 CreateThread API创建后台存档线程。线程函数每60秒循环检查volatile标志位，若游戏已初始化则自动调用SaveManager保存玩家数据。程序退出时通过WaitForSingleObject等待线程安全结束。

7.4 等级自动成长

checkLevelUp()函数在每次获得经验后调用，使用while循环支持连续升级：检查EXP阈值 → 扣除阈值经验 → 等级+1 → HP上限+20、ATK+5 → 满血恢复 → 重复检测。经验阈值公式为 $\text{expToNextLevel}(\text{lv}) = \text{lv} * 30 + 20$ 。

7.5 编码转换方案（技术创新）

本项目遇到的核心技术挑战是UTF-8编码与Win32 W-API之间的中文转换。通过修改U2W/W2U函数从CP_ACP改为CP_UTF8编码页，配合编译选项-fexec-charset=UTF-8，实现了窄字符串（UTF-8）与宽字符串（UTF-16）的正确互转，彻底解决了中文乱码问题。

八、测试与结果分析

8.1 测试策略

本项目采用手动功能测试为主、编译期静态检查为辅的测试策略。每个模块开发完成后进行黑盒功能测试，覆盖正常流程、边界条件和异常场景。

8.2 测试用例与结果

模块	测试用例	预期结果	状态
角色管理	创建角色输入中文名称	成功创建，名称正确显示	通过
角色管理	查看属性面板	等级Lv. 1，HP/MP正确	通过
背包管理	打开空背包	显示“背包是空的”	通过
商店系统	购买校园面包 (15G)	金币-15，背包+1	通过
商店系统	金币不足时购买	提示“金币不足”	通过
商店系统	出售物品（半价回收）	金币+7，物品消失	通过
战斗系统	挑战懒散学渣小怪	胜利获得30EXP+15金币	通过
战斗系统	挑战期末压力Boss	奖励150EXP+90金币	通过
等级成长	累积足够EXP升级	自动升级，HP+20，ATK+5	通过
任务系统	完成“新手初试冒险”	战斗1场后可领奖	通过
存档系统	保存后关闭再打开	数据正确恢复	通过
中文显示	物品名/怪物名/任务名	中文无乱码	通过

8.3 关键问题与解决方案

中文乱码问题：编译选项`-fexec-charset=UTF-8`使窄字符串编码为UTF-8，但U2W等转换函数使用CP_ACP(GBK)。解决方案：修改三处转换函数，将CP_ACP改为CP_UTF8。

调试对话框bug：商店初始化代码中含类型不匹配的`wsprintfW(%s传入char*)`调试消息框。解决方案：移除调试代码。

MinGW无mingw32-make：CMake配置时缺少构建工具。解决方案：改为直接使用g++命令行编译，更新CMakeLists.txt补充所有源文件。

L"" vs narrow string: 硬编码宽字符串(L"中文")可正常显示，但窄字符串需通过U2W转换。确认了统一使用UTF-8编码策略。

九、总结与展望

9.1 项目总结

本项目成功实现了一个功能完整的校园RPG冒险游戏，完整覆盖了课程要求的六大核心模块，并额外完成了多项挑战任务。通过本项目实践，团队成员深入理解了以下C++核心概念：

- 继承与多态：通过Item和Skill两大多态继承体系，实践了纯虚函数、虚函数表、运行时类型识别（RTTI）等机制。
- STL容器应用：大量使用vector管理动态对象集合，string处理文本，sstream进行格式化输出和序列化。
- 设计模式：工厂模式（SkillFactory）、原型模式（Item::clone()）、策略模式（不同Skill的calcDamage算法）、模板方法模式（序列化接口）。
- Win32编程：学习了Windows消息循环、窗口过程、对话框模板、资源文件（.rc）、GDI字体渲染等。
- 编码与国际化：深入理解了UTF-8/GBK/UTF-16编码差异和MultiByteToWideChar编码页转换机制。
- 多线程编程：实践了Win32 CreateThread创建后台线程、volatile标志位线程通信、WaitForSingleObject线程同步。

9.2 未来展望

- 接入SQLite数据库：将道具数据、任务模板、怪物数据迁移至SQLite，实现结构化数据管理。
- 音效与背景音乐：添加WAV/MP3音效支持，增强游戏沉浸感。
- 地图探索系统：实现校园场景地图，不同区域分布不同怪物和NPC。
- 装备强化系统：支持装备升级、附魔、套装属性等进阶玩法。
- 网络排行榜：接入简单HTTP服务器，实现玩家等级和战斗排名。
- 跨平台支持：使用SDL2或Qt框架替代Win32 API，实现Linux/macOS兼容。

十、参考文献

- [1] Bjarne Stroustrup. The C++ Programming Language (4th Edition). Addison-Wesley, 2013.
- [2] Stanley B. Lippman et al. C++ Primer (5th Edition). Addison-Wesley, 2012.
- [3] Charles Petzold. Programming Windows (5th Edition). Microsoft Press, 1998.
- [4] Microsoft Docs. MultiByteToWideChar function. docs.microsoft.com
- [5] Microsoft Docs. DialogBoxW macro. docs.microsoft.com
- [6] Erich Gamma et al. Design Patterns. Addison-Wesley, 1994.
- [7] Scott Meyers. Effective Modern C++. OReilly Media, 2014.
- [8] MinGW-w64 Project. GCC Compiler Manual. gcc.gnu.org
- [9] CMake Documentation. cmake.org/documentation/
- [10] The Unicode Consortium. Unicode Standard, Version 15.0. unicode.org

感谢阅读